

# injectService

```
readonly navigation = injectService(  
  Router,  
  ({ navigateByUrl, currentNavigation }) => ({  
    decline: () => navigateByUrl('/terms/declined'),  
    isNavigating: computed(  
      () => currentNavigation() !== null  
    ),  
  }),  
);
```



**@craft-ng - Type-Safe Reactive States  
Management for Angular**  
Promotes declarative and composable  
code with logic management utilities  
Par Romain Geffrault



# injectService

Créez des façades typées sur vos services Angular

**Le problème :** Tu injectes un service, tu utilises 3 méthodes sur 15, et tout le reste fuite dans ton composant.

**La solution :** `injectService` te laisse choisir exactement ce que tu exposes.

Même pattern que `state`, `query`, `mutation` : des insertions composables.



# L'exemple : une façade Router

```
import { Component, computed } from '@angular/core';
import { Router } from '@angular/router';
import { injectService, on$, source$ } from '@craft-ng/core';

@Component({
  selector: 'app-terms-page',
  template: '',
  standalone: true,
})
export class TermsPageComponent {
  private readonly userAccept = source$<void>();

  readonly navigation = injectService(
    Router,
    ({ navigateByUrl, currentNavigation }) => ({
      decline: () => navigateByUrl('/terms/declined'),
      navigateOnAccept: on$(this.userAccept, () =>
        navigateByUrl('/checkout/shipping',
          { replaceUrl: true })),
    }),
    isNavigating: computed(
      () => currentNavigation() !== null
    ),
  ),
};
```

navigation n'expose que decline, navigateOnAccept et isNavigating.

navigateOnAccept est un binding interne (on\$) filtré du résultat.



# Des insertions chaînables

Chaque insertion accède aux résultats des précédentes

```
readonly checkout = injectService(
  CheckoutService,
  ({ cart, status, total, submitOrder }) => ({
    total,
    status,
    itemCount: computed(() =>
      cart().reduce(
        (count, item) => count + item.quantity, 0
      ),
    ),
    submit: submitOrder,
  }),
  ({ insertions }) => ({
    canSubmit: computed(() =>
      insertions.itemCount() > 0
      && insertions.status() === 'editing',
    ),
    summaryLabel: computed(() =>
      `${insertions.itemCount()} items`
      + ` - ${insertions.total()} EUR`,
    ),
  }),
);
```

La 2e insertion utilise `insertions.itemCount()` et `insertions.status()` définis juste avant.





# Le même pattern que les primitives

injectService suit la même logique que toutes les primitives @craft-ng

```
primitive(  
  config,          // Configuration de base  
  insertion1,      // Méthodes / computed  
  insertion2,      // Dérivations  
  insertion3,      // Side effects  
);  
  
injectService(  
  ServiceToken,    // Service à injecter  
  insertion1,      // API exposée  
  insertion2,      // Dérivations  
  insertion3,      // Side effects  
);
```

- ✓ Méthodes du service bindées automatiquement
- ✓ Bindings internes (on\$, afterRecomputation) filtrés du résultat
- ✓ Tout est typé de bout en bout



# Ce qui arrive

**Disponible dans la prochaine version** de @craft-ng (après la finalisation du signalForm)

## Roadmap :

- gestion des options d'injection (optional, host, self...)
- encore plus de flexibilité sur les insertions

## Cas d'usage idéaux :

- Façades sur services tiers (Router, FormBuilder...)
- Composant orchestrateur de logique métier
- Exposition contrôlée d'une API de service

📖 Doc : <https://ng-angular-stack.github.io/craft/>



**Merci pour la lecture !  
Dis-moi ce que t'en  
penses en commentaire.**



**Romain Geffrault**  
**Développeur Angular**  
Suis moi pour du contenu dédié à  
Angular